NYCU Pattern Recognition, Homework 3

109550071, 丁宇洋

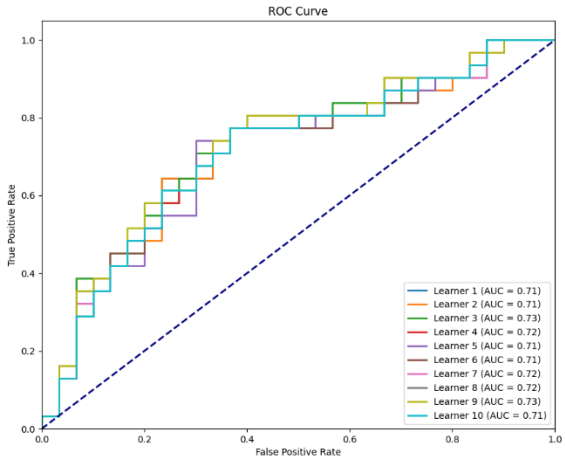
Part. 1, Coding (60%):

(20%) Adaboost

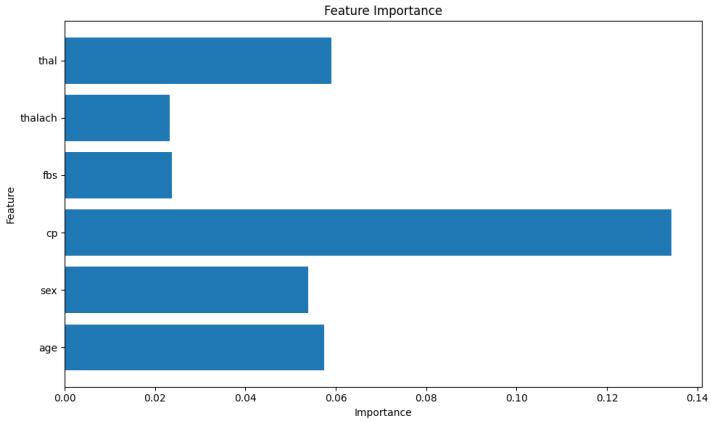
1. (10%) Show your accuracy of the testing data (n_estimators = 10)

```
2024-05-15 11:32:44.777 | INFO     |  main  :main:86 - AdaBoost - Accuracy: 0.7016
```

2. (5%) Plot the AUC curves of each weak classifier.



3. (5%) Plot the feature importance of the AdaBoost method. Also, you should snapshot the implementation to calculate the feature importance.



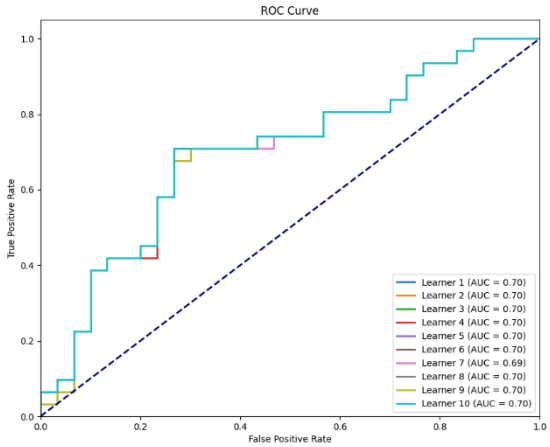

```python
def compute_feature_importance(self) -> t.Sequence[float]:
    input_dim = self.learners[0].fc1.weight.size(1)
    feature_importance = torch.zeros(self.learners[0].input_dim)
    for alpha, model in zip(self.alphas, self.learners):
        feature_importance += alpha * torch.sum(torch.abs(model.fc1.weight.data), dim=0)
    return feature_importance.detach().cpu().numpy()
```

(20%) Bagging

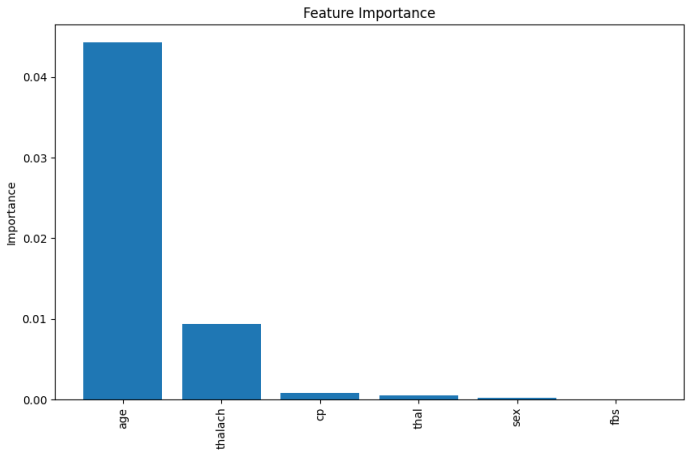
4. (10%) Show your accuracy of the testing data with 10 estimators. (n_estimators=10)

```
2024-05-15 11:36:01.006 | INFO     | __main__:main:108 - Bagging - Accuracy: 0.4966
```

5. (5%) Plot the AUC curves of each weak classifier.



6. (5%) Plot the feature importance of the Bagging method. Also, you should snapshot the implementation to calculate the feature importance.




```python
def compute_feature_importance(self, X_test, y_test) -> t.Sequence[float]:

    X_test_tensor = torch.from_numpy(X_test).float()
    y_test_tensor = torch.from_numpy(y_test).float()
    criterion = nn.BCEWithLogitsLoss()

    baseline_scores = []
    for model in self.learners:
        with torch.no_grad():
            outputs = model(X_test_tensor)
            loss = criterion(outputs.squeeze(), y_test_tensor)
            baseline_scores.append(loss.item())

    feature_importances = np.zeros(X_test.shape[1])
    for i in range(X_test.shape[1]):
        permuted_X_test = X_test.copy()
        np.random.shuffle(permuted_X_test[:, i])
        permuted_X_test_tensor = torch.from_numpy(permuted_X_test).float()

        scores = []
        for model in self.learners:
            with torch.no_grad():
                outputs = model(permuted_X_test_tensor)
                loss = criterion(outputs.squeeze(), y_test_tensor)
                scores.append(loss.item())

        feature_importances[i] = np.mean(scores) - np.mean(baseline_scores)

    return feature_importances
```

(15%) Decision Tree

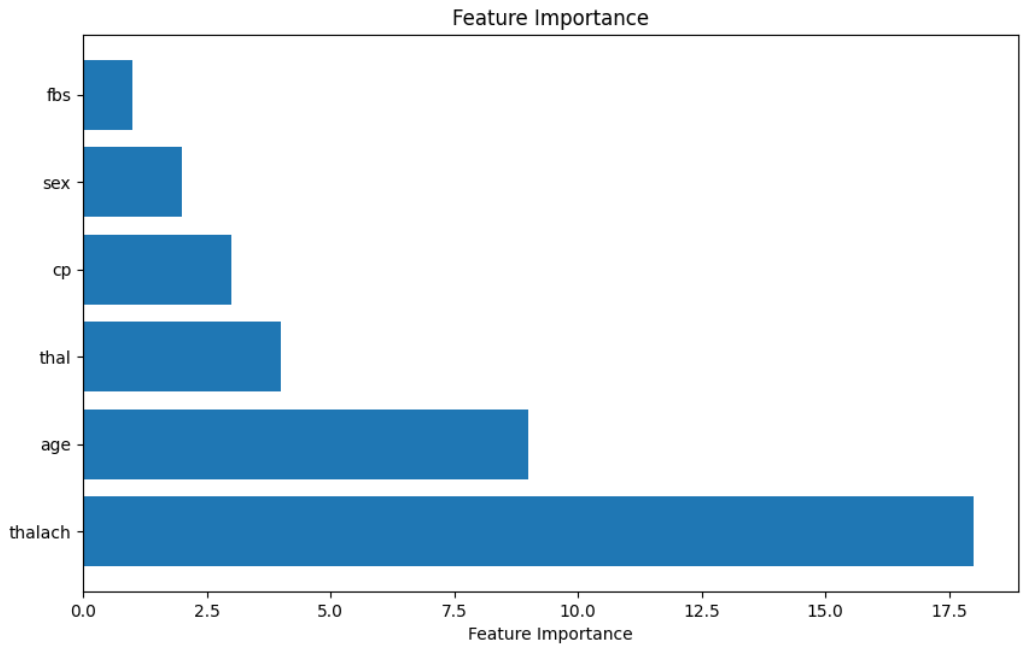
7. (5%) Compute the gini index and the entropy of the array [0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1].

```
Gini of data is  0.4628099173553719
Entropy of data is  0.9456603046006401
```

8. (5%) Show your accuracy of the testing data with a max-depth = 7

```
2024-05-15 11:36:16.246 | INFO     | __main__:main:122 - DecisionTree - Accuracy: 0.7377
```

9. (5%) Plot the feature importance of the decision tree.



(5%) Code Linting

10. Show the snapshot of the flake8 linting result.

```
PS C:\Users\user\this_semester\pr\hw3\release> flake8 main.py
PS C:\Users\user\this_semester\pr\hw3\release>
```

Part. 2, Questions (40%):

1. (10%) We have three distinct binary classifiers, and our goal is to leverage them in creating an ensemble classifier through the majority voting strategy to make decisions.
   Assuming each individual binary classifier operates independently of the others with an accuracy of 60%, what would be the accuracy of the ensemble classifier?
   Since each classifier operates independently with an accuracy of 60%, the probability of each event is: $P(A) = P(B) = P(C) = 0.6$, i.All three classifiers predict correctly: $P(A \cap B \cap C) = 0.6 \times 0.6 \times 0.6 = 0.216$, ii. Two classifiers predict correctly, and one predicts incorrectly: $P(A \cap B \cap C') + P(A \cap B' \cap C) + P(A' \cap B \cap C) = 3 \times (0.6$

× 0.6 × 0.4) = 0.432, 0.216 + 0.432 = 0.648. The accuracy of the ensemble classifier is 64.8%.

2. (15%) For the decision tree algorithm, we can use the "pruning" technique to avoid overfitting. Does the random forest algorithm also need pruning?
Please explain in detail.
No, the random forest algorithm doesn't require pruning to avoid overfitting. The random forest algorithm has other mechanism(such as ensemble effect, bootstrap aggregating, feature subsampling and deep trees) that can help to prevent overfitting, so it doesn't need pruning.

3. (15%) Activation functions are core components of neural networks. They need to be differentiable to ensure backpropagation works correctly. Please calculate the derivatives of the following commonly used activation functions.
(**For questions 1. and 2., consider the cases where $x > 0$ and $x \leq 0$**)

1. f(x) = relu(x),	df(x)/dx = ?
2. f(x) = leaky_relu(x) with negative_slope=0.01,	df(x)/dx = ?
3. f(x) = sigmoid(x),	df(x)/dx = ?
4. f(x) = silu(x),	df(x)/dx = ?
5. f(x) = tanh(x),	df(x)/dx = ?

1.
Case1:x>0:
relu(x) = x, df(x)/dx = d(x)/dx = 1
Case2:x<=0:
relu(x)=0, df(x)/dx = d(0)/dx = 0

2.
Case1:x>0:
leaky_relu (x) = x, df(x)/dx = d(x)/dx = 1
Case2:x<=0:
leaky_relu (x)= 0.01x, df(0.01x)/dx = 0.01

3.
Let y = sigmoid(x), dy/dx = (d/dx)(1 / (1 + e^(-x))) = (e^(-x)) / (1 + e^(-x))^2
= (1 / (1 + e^(-x))) * (e^(-x) / (1 + e^(-x))) = sigmoid(x) * (1 - sigmoid(x))

4.
Let y = silu(x) = x * sigmoid(x) dy/dx = x * d(sigmoid(x))/dx + sigmoid(x) * d(x)/dx = x * sigmoid(x) * (1 - sigmoid(x)) + sigmoid(x) = sigmoid(x) * (x + x * (1 - sigmoid(x))) = sigmoid(x) * (x + sigmoid(x) - x * sigmoid(x)) = sigmoid(x) * (x + sigmoid(x) * (1 - x))
5.
Let y = tanh(x)
dy/dx = (d/dx)((e^x - e^(-x)) / (e^x + e^(-x)))
quotient rule:
dy/dx = (e^x + e^(-x)) * (e^x - e^(-x))' - (e^x - e^(-x)) * (e^x + e^(-x))'

(e^x + e^(-x))^2
simplifying:
dy/dx = (e^x + e^(-x)) * (e^x + e^(-x)) - (e^x - e^(-x)) * (e^x - e^(-x))

(e^x + e^(-x))^2
dy/dx = 2 * (e^(2x) + 1) - 2 * (e^(2x) - 1)

(e^x + e^(-x))^2
dy/dx = 4 * e^(2x)

(e^x + e^(-x))^2
dy/dx = 4 * (e^x)^2 / (e^x + e^(-x))^2
= 4 * (tanh(x))^2 / (e^x + e^(-x))

Substituting (e^x + e^(-x)) = 1 / (cosh(x)),
get:
dy/dx = 4 * (tanh(x))^2 / (1 / cosh(x))
= 4 * (tanh(x))^2 * cosh(x)
= sech^2(x)
Finally, df(x)/dx = sech^2(x) = (1 - tanh^2(x))